
Where Spark's Advantage Comes From: Hadoop MapReduce and Spark on a Two-Node EC2 Cluster

Huacong Tang
University of Southern California

Xiao Yang
University of Southern California

Jiachen Lu
University of Southern California

[Code](#) [Demo Video](#)

1 Testbed

Two EC2 `c7i-flex.large` instances (2 vCPU, 4 GiB, 40 GB gp3) in `us-west-2`, Ubuntu 22.04, JDK 11, Hadoop 3.3.6, Spark 3.5.1, HDFS replication 2, YARN NodeManager 3072 MB / 2 vcores, 1024 MB containers. Spark runs on YARN, not in local mode, so the second node actually participates. Two Project Gutenberg corpora: **small**, 6 books, 3.7 MB; **large**, 1.1 GB concatenated into ten 128 MB blocks. Timings are the application's own elapsed time from the YARN ResourceManager REST API, the same source for both frameworks; each cell is the mean of three runs (two on the large corpus) with the half-range shown. Every number traces to `bench/matrix.sh` and the CSV it wrote.

We report the instance type as a deviation: the handout recommends `t2.medium`, a burstable type whose throttling after credit exhaustion would corrupt precisely the quantity this lab measures.

2 Results

Table 1: Application elapsed time (s), mean $\pm$ half-range. MapReduce minmax is charged for both passes: as run it is a second job over the wordcount output, while Spark's recomputes the counts from the raw text, so the single-pass number (27.9 s on 1.1 GB) answers a different question.

Corpus	Job	MapReduce		Spark on YARN	
		1 node	2 nodes	1 node	2 nodes
small	wordcount	31.8 $\pm$ 1.3	22.1 $\pm$ 2.2	35.6 $\pm$ 1.0	17.1 $\pm$ 4.0
small	charcount	33.9 $\pm$ 0.1	24.8 $\pm$ 1.6	36.3 $\pm$ 0.8	14.7 $\pm$ 0.6
small	minmax	49.8 $\pm$ 1.8	40.5 $\pm$ 2.9	35.9 $\pm$ 0.4	15.1 $\pm$ 0.3
large	wordcount	408.0 $\pm$ 6.6	294.1 $\pm$ 14.6	171.1 $\pm$ 0.4	91.8 $\pm$ 0.7
large	charcount	1894.4 $\pm$ 19.7	1353.7 $\pm$ 85.8	449.2 $\pm$ 0.6	246.6 $\pm$ 0.5
large	minmax	436.0 $\pm$ 7.3	324.3 $\pm$ 15.2	172.9 $\pm$ 0.9	93.2 $\pm$ 1.4

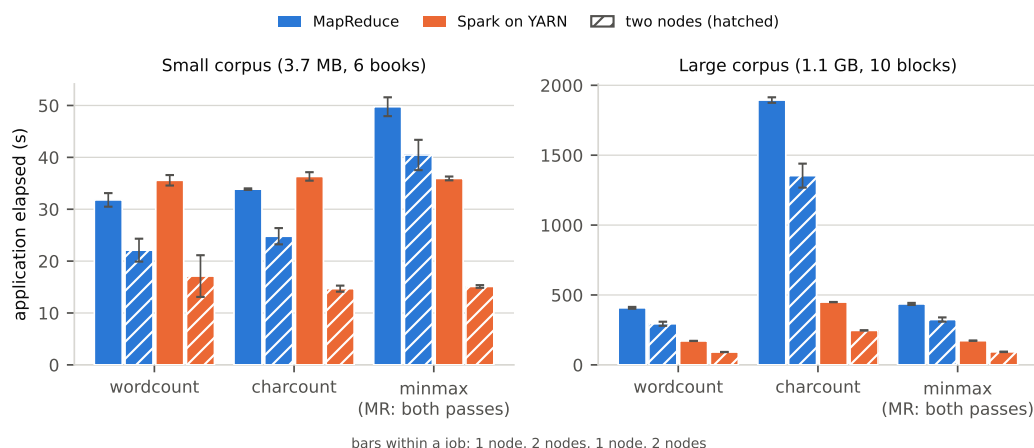


Figure 1: The same measurements. Note the two panels have different scales: the small corpus is dominated by fixed startup cost, the large one by work.

3 Answers

Which framework executed faster on one node? On two nodes? On one node with the small corpus the two are effectively tied, and MapReduce is even marginally ahead on the single-pass jobs (31.8 s vs. 35.6 s for wordcount), because at 3.7 MB both are measuring startup rather than work. Everywhere else Spark wins: on 1.1 GB it is 2.4–4.2 $\times$ faster on one node and 3.2–5.5 $\times$ faster on two. Spark also scales better across the node count (1.86 $\times$ on wordcount against MapReduce’s 1.39 $\times$), so its lead widens rather than narrows as the cluster grows.

Which framework is easier to implement in Python? Spark, and the gap is structural rather than stylistic: each job is one script of about ten lines, against a mapper, a reducer, and a Streaming command line whose `-files` and `-numReduceTasks 1` flags are both silent failure modes if omitted. Min/max is the clearest case, needing two programs and a second job over the first job’s output under MapReduce because a mapper emitting `word\t1` makes every extremum 1. Spark has its own traps, notably that the master must come from `spark-submit` and that paths need an explicit `hdfs://` scheme, but neither changes the program’s shape.

How does Spark’s in-memory processing affect performance compared to Hadoop? Measured directly on the cached word-count RDD at 1.1 GB, the first action took 139.8 s and the second 4.0 s, a 35 $\times$ difference for the same data, because only the first pays for the shuffle. That mechanism is what removes MapReduce’s obligation to write intermediate results to HDFS between passes, and it is why Spark’s advantage grows with data size rather than staying constant. It is not the whole story, though: JVM reuse across tasks and the absence of a per-task Python process contribute to the same gap, so “in-memory” names one cause among several rather than the cause.